

Homework 1

Dávid Straka

February 2024

(a)

I would implement the following algorithm. Let c be a counter of times that the average of rewards from action 1 was greater than the average of rewards from action 2. Set $n = 0$.

1. Increase n by one.
2. Choose action 1.
3. Choose action 2.
4. Determine averages of rewards from action 1 and action 2, i.e.,

$$\bar{X}_1 := \frac{1}{n} \sum_{t:A_t=1} X_t \quad \bar{X}_2 := \frac{1}{n} \sum_{t:A_t=2} X_t.$$

5. If $\bar{X}_1 > \bar{X}_2$, increase c by one. If $\bar{X}_1 < \bar{X}_2$, decrease c by one.
6. If $c \geq a$, say $a = 5$, terminate this algorithm and choose only actions 1. If $c \leq -a$ terminate this algorithm and choose only actions 2. Otherwise, return to Step 1.

The core of this policy is to obtain some certainty of the reward expectation of each action. We take both actions in each iteration of the algorithm above. We estimate the reward expectation of action 1 and 2 by sample means of the obtained rewards.

We want some certainty in claiming that one sample mean of rewards is actually greater than the other. Hence, the counter c_t checks whether at iteration t for at least a and at most $2a$ iterations, one of the sample means is greater than the other. Hence, for example, if the sample mean of rewards w.r.t. action 1 is consistently greater than the mean w.r.t. action 2, we choose action 1 for the rest of the policy.

The more one wants to be certain about choosing the action with higher reward, the greater the value of a one chooses. However, the DM focuses on maximizing the reward. A great value of a undermines this because it possibly infinitely prolongs the explorative averaging phase.

Hence, formally, the action

$$A_t := \begin{cases} 1, & \text{if } |c_t| < 5 \text{ and } t \text{ is odd} \\ 2, & \text{if } |c_t| < 5 \text{ and } t \text{ is even} \\ 1 & \text{if } c_t \geq 5 \\ 2 & \text{if } c_t \leq -5. \end{cases}$$

(b)

For example, say that we want to run the algorithm for $X_1 \sim U[0.1, 0.3]$ and $X_2 \sim U[0.1, 0.5]$. We know that the expected values are 0.2 and 0.3, respectively. Since we run the algorithm 200 times, we obtain the optimal expected reward $200 * 0.3 = 60$.

We want to determine the instance-depending regret, with confidence intervals, as defined in the notes. Every time we run the algorithm we choose action 1 a_1 -times and action 2 a_2 -times. We can obtain the instance-depending regret by $R = 60 - a_1 * 0.2 - a_2 * 0.3$. For this example, we obtain regret 10 using the attached Python code.

If we run the simulation 1000-times, we can obtain a confidence interval by taking the 2.5-th and 97.5-th percentile of the simulated regrets. We obtain CI (2.5, 17.5) using the attached Python code.

Python Code

Listing 1: Python Code

```
import numpy as np

class AverageComparisonBandit:
    def __init__(self, num_thresh, num_runs, l1, u1, l2, u2):
        self.num_thresh = num_thresh # policy threshold, number a
        self.num_runs = num_runs # n = number of time periods
        self.l1 = l1
        self.u1 = u1
        self.l2 = l2
        self.u2 = u2
        self.exp_1 = (l1 + u1)/2 # expected reward of action 1
        self.exp_2 = (l2 + u2)/2 # expected reward of action 2
        self.opt_exp = max(self.exp_1, self.exp_2) # optimal expected reward

        self.regrets = []
        self.CI = []

        self.time = 0
```

```

        self.average1 = 0 # average reward from action 1
        self.average2 = 0 # average reward from action 2
        self.counter = 0 # counts times that average_0 > average_1
        self.history = []
#         self.reward = 0
        self.regret = 0
        self.num_action1 = 0 # how many times has action 1 occurred
        self.num_action2 = 0 # how many times has action 2 occurred

    def do_both_actions(self):
        self.do_action1()
        self.do_action2()

    def do_action1(self):
        self.time += 1
        reward = np.random.uniform(self.l1, self.u1)
        self.history.append([1, reward])

    def do_action2(self):
        self.time += 1
        reward = np.random.uniform(self.l2, self.u2)
        self.history.append([2, reward])

    def get_reward_means(self):
        self.average1 = np.mean([l[1] for l in self.history if l[0] == 1])
        self.average2 = np.mean([l[1] for l in self.history if l[0] == 2])

    def do_policy(self):
        if self.counter == self.num_thresh:
            self.do_action1()
        elif self.counter == - self.num_thresh:
            self.do_action2()
        else:
            self.do_both_actions()

        self.get_reward_means()

    def update_counter(self):
        if abs(self.counter) == 5:
            pass
        elif self.average1 > self.average2:
            self.counter += 1
        else:
            self.counter -= 1

#     def get_reward(self):

```

```

#         self.reward = np.sum([k[1] for k in self.history[:self.num_runs]])

def run_algorithm(self):
    while self.time < self.num_runs:
        self.do_policy()

        self.get_regret()

def get_regret(self):
    self.num_action1 = len([1 for l in self.history[:self.num_runs] if l[0] == 1])
    self.num_action2 = self.num_runs - self.num_action1
    self.regret = self.num_runs * self.opt_exp - self.num_action1 * self.exp

def reset(self):
    self.time = 0
    self.average1 = 0 # average reward from action 1
    self.average2 = 0 # average reward from action 2
    self.counter = 0 # counts times that average_0 > average_1
    self.history = []
#     self.reward = 0
    self.regret = 0
    self.num_action1 = 0 # how many times has action 1 occurred
    self.num_action2 = 0 # how many times has action 2 occurred

def get_confidence_interval(self):

    self.regrets = []
    for _ in range(1000):
        self.reset()
        self.run_algorithm()
        self.regrets.append(self.regret)

    self.regret = np.mean(self.regrets)
    self.CI = np.percentile(self.regrets, [2.5, 97.5])

num_thresh = 5
num_runs = 200
bandit = AverageComparisonBandit(num_thresh, num_runs, 0.3, 0.5, 0.7, 0.9)

bandit.get_confidence_interval()
print(bandit.regret)
print(bandit.CI)

```